

SISTEMA DE GESTIÓN BUMAND

Fundación Diaconía FRIF-IFD

MANUAL DE ARQUITECTURA TÉCNICA Y ESPECIFICACIÓN DEL BACKEND

Especificación Exhaustiva del Monolito Modular NestJS, Modelo Relacional de 15
Tablas y Reglas de Negocio Institucionales

Módulo de Software:	bumand-backend
Arquitectura:	Monolito Modular con Inyección de Dependencias
Framework Base:	NestJS 11 & TypeScript 5.9
Motor de Persistencia:	TypeORM 1.1 / PostgreSQL 15+
Seguridad y Control de Acceso:	Passport JWT, RBAC por Metadatos, Bcrypt (10 rondas)
Motor de Reportes Oficiales:	Puppeteer Headless PDF Engine
Gestión Académica:	2026

Índice

1. Fundamentos Arquitectónicos y Filosofía del Sistema	2
2. Estructura del Código Fuente (src/)	2
3. Núcleo del Servidor y Configuración de Entrada	3
3.1. Punto de Entrada (src/main.ts)	3
3.2. Módulo Raíz (src/app.modulo.ts)	3
4. Utilidades Transversales y Algoritmos Canónicos	3
4.1. Geolocalización Satelital: Fórmula de Haversine (geo.utilidad.ts)	3
4.2. Seguridad Criptográfica (hash.utilidad.ts)	3
5. Persistencia, Modelo Relacional de 15 Tablas y Semilla	3
5.1. Tipos Enumerados del Motor (ENUMs)	3
5.2. Diccionario de las 15 Tablas Relacionales	4
5.3. Datos Semilla Base (semilla.servicio.ts)	4
6. Módulos de Dominio y Lógica de Negocio	4
6.1. Autenticación y Control de Acceso (RBAC)	4
6.2. Gestión de Usuarios y Becarios	5
6.3. Control de Asistencia y Geolocalización	5
6.4. Liquidación de Pasajes: Centavos Enteros y Regla del Día 24	5
6.5. Evaluación Integral 360° y Formulario F-03	5
6.6. Prácticas Preprofesionales y Firma Digital	6
6.7. Dashboard Analítico y Notificaciones	6
6.8. Motor de Emisión de Reportes PDF (pdf)	6
7. Matriz de Endpoints REST de la API	6
8. Verificación de Calidad y Pruebas Automatizadas	6

1 Fundamentos Arquitectónicos y Filosofía del Sistema

El ecosistema tecnológico BUMAND constituye la plataforma centralizada para el seguimiento académico, control de asistencia, liquidación presupuestaria y evaluación del Programa de Becarios Universitarios de la **Fundación Diaconía Fondo Rotativo de Inversión y Fomento (Diaconía FRIF-IFD)**.

Históricamente, la gestión diaconal de estudiantes becarios dependió de instrumentos analógicos: planillas de asistencia físicas, rendiciones de gastos de transporte con cálculos empíricos y formularios pastorales manuscritos. Dicha dinámica generaba demoras administrativas, inconsistencias en el reembolso del 80 % de pasajes y pérdida de trazabilidad histórica.

Para transformar este escenario con rigor de ingeniería de software, el backend de BUMAND ha sido implementado como un **monolito modular desacoplado** basado en **NestJS 11** y **TypeScript 5.9**, soportado por el ORM **TypeORM 1.1** y una base de datos relacional **PostgreSQL 15+**. La arquitectura se rige por tres pilares:

1. **Inversión de Control y Tipado Estricto:** Desacoplamiento funcional mediante inyección de dependencias en constructores y tipado riguroso sin uso de `any`.
2. **Estándar de Español Absoluto:** Correspondencia semántica total entre la normativa institucional y el código fuente (clases en PascalCase, métodos en camelCase, atributos de base de datos en snake_case, archivos y rutas en kebab-case).
3. **Seguridad Perimetral y Validación Estricta:** Filtrado de parámetros no reconocidos mediante `ValidationPipe` global, autenticación sin estado con JWT y control de acceso basado en roles (RBAC).

2 Estructura del Código Fuente (src/)

El árbol de carpetas refleja la separación entre núcleo, infraestructura y módulos de dominio:

```
bumand-backend/src/
|-- main.ts                # Entrada y ValidationPipe global
|-- app.modulo.ts          # Modulo raiz y conexion TypeORM
|-- app.controlador.ts     # Endpoint raiz de verificacion
|-- app.servicio.ts        # Servicio basico de salud
|-- utilidades/            # Utilidades transversales
|   |-- geo.utilidad.ts    # Formula de Haversine (geocercas)
|   \-- hash.utilidad.ts   # Bcrypt con 10 rondas de salting
|-- datos/                 # Infraestructura de persistencia
|   |-- 001_esquema_completo_15_tablas.sql # DDL de 15 tablas en PostgreSQL
|   |-- datos.modulo.ts    # Modulo TypeORM para datos
|   \-- semilla.servicio.ts # Seeding automatico de datos
\-- modulos/               # Modulos de dominio funcional
    |-- autenticacion/     # Tokens JWT y guardias RBAC
    |-- usuarios/          # Gestion de cuentas y estados
    |-- becarios/          # Directorio y ficha academica
    |-- iglesias/          # Congregaciones eclesiasiticas
    |-- lugares-practica/   # Sedes fisicas y geocercas
    |-- asistencia/        # Marcacion GPS y computo de horas
    |-- pasajes/           # Calculo del 80% y regla del dia 24
    |-- evaluaciones/      # Matriz 360 grados y Formulario F-03
    |-- practicas/         # Convalidacion y firma digital
    |-- notificaciones/    # Alertas y mensajeria FCM
    \-- dashboard/        # Analitica de horas y presupuesto
```

\-- pdf/

Motor Puppeteer de reportes

3 Núcleo del Servidor y Configuración de Entrada

3.1 Punto de Entrada (src/main.ts)

El archivo `main.ts` inicializa la aplicación mediante `NestFactory.create(AppModulo)` y establece la tubería global de transformación y validación:

- `whitelist: true`: Descarta automáticamente campos no declarados en los DTOs.
- `forbidNonWhitelisted: true`: Lanza una excepción `BadRequestException` si se reciben parámetros espurios.
- `transform: true`: Realiza la conversión automática de tipos primitivos hacia las clases tipadas.

3.2 Módulo Raíz (src/app.modulo.ts)

Configura el módulo de entorno `ConfigModule.forRoot({ isGlobal: true })` y la conexión relacional con `TypeOrmModule.forRoot`. Registra de forma explícita las **15 entidades relacionales** y establece `synchronize: false` para asegurar que las modificaciones de base de datos se ejecuten exclusivamente mediante migraciones SQL controladas.

4 Utilidades Transversales y Algoritmos Canónicos

4.1 Geolocalización Satelital: Fórmula de Haversine (geo.utilidad.ts)

Para comprobar si la marcación de un estudiante se realiza dentro de la geocerca de su lugar de práctica, el sistema aplica la fórmula trigonométrica de Haversine:

$$\Delta\varphi = (\text{lat}_2 - \text{lat}_1) \cdot \frac{\pi}{180}, \quad \Delta\lambda = (\text{lon}_2 - \text{lon}_1) \cdot \frac{\pi}{180} \quad (1)$$

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2)$$

$$c = 2 \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right), \quad d = R \cdot c \quad (3)$$

Donde $R = 6,371,000$ m es el radio medio de la Tierra. La función retorna la distancia en metros con precisión centimétrica.

4.2 Seguridad Criptográfica (hash.utilidad.ts)

El resguardo de credenciales utiliza `bcrypt` con 10 rondas de salting:

- `hashearContrasena(contrasena)`: Genera un resumen criptográfico irreversible con sal aleatoria.
- `compararContrasena(contrasena, hash)`: Comprobación en tiempo constante para mitigar ataques de temporización.

5 Persistencia, Modelo Relacional de 15 Tablas y Semilla

5.1 Tipos Enumerados del Motor (ENUMs)

El archivo `001_esquema_completo_15_tablas.sql` define ocho tipos enumerados nativos para garantizar la integridad de dominio:

- **rol_usuario:** Roles de usuario (*becario, supervisor, administrador*).
- **estado_usuario:** Estado de cuenta (*activo, inactivo*).
- **tipo_asistencia:** Tipos de jornada (*practicas, escuela_lideres, acciones_servicio*).
- **estado_solicitud:** Flujo de pasajes (*borrador, pendiente, aprobado, rechazado*).
- **tramo_recorrido:** Sentido del viaje (*ida, vuelta*).
- **tipo_evaluador:** Actores (*pastor, becario, supervisor, facilitador, trabajador_social*).
- **estado_periodo:** Vigencia (*abierto, cerrado*).
- **estado_evaluacion:** Fase (*pendiente, completado*).

5.2 Diccionario de las 15 Tablas Relacionales

1. **usuarios:** Credenciales del sistema, correo único, clave encriptada, rol institucional, estado y token FCM.
2. **iglesias:** Congregaciones cristianas de origen de los estudiantes becarios.
3. **lugares_practica:** Sedes físicas de Diaconía, coordenadas decimales y radio métrico de tolerancia.
4. **becarios:** Ficha institucional y académica vinculada 1:1 con la cuenta de usuario.
5. **registros_asistencia:** Marcaciones horarias de ingreso y salida con validación de geocerca satelital.
6. **solicitudes_pasajes:** Cabecera mensual de viáticos con cálculo de liquidación al 80 %.
7. **recorridos:** Trayectos de transporte individuales con origen, destino, tarifa y justificación.
8. **modulos_evaluacion:** Catálogo de los cinco ejes formativos del Programa BUMAND.
9. **periodos_evaluacion:** Ciclos semestrales con fechas de vigencia y control de admisión.
10. **evaluadores:** Catálogo multi-actor de los evaluadores asignados al programa.
11. **evaluaciones:** Calificaciones por módulo y periodo con restricción de unicidad y nota sobre 100.
12. **evaluaciones_pastor:** Extensión 1:1 eclesiástica para el Formulario F-03 con campos en JSONB.
13. **aprobaciones_practicas:** Certificación oficial de horas y resguardo de la firma digital.
14. **notificaciones:** Mensajes internos y alertas reactivas para los usuarios del sistema.
15. **reportes_generados:** Registro histórico y auditoría de los documentos PDF emitidos.

5.3 Datos Semilla Base (`semilla.servicio.ts`)

Al iniciar el servidor (`OnModuleInit`), el servicio siembra automáticamente:

- **Administrador General:** Cuenta institucional creada con contraseña encriptada y rol ADMINISTRADOR.
- **Sede Central El Alto:** Ubicada en Av. Juan Pablo II con latitud -16.505000 , longitud -68.163000 y radio de 50 m.
- **Tres Becarios Institucionales:** Nilda Churata (Seguridad Física), Edgar Alarcón (Consultorio Médico) y Adai Huayta (Productos y Canales).
- **Catálogo de Evaluación:** Los 5 módulos canónicos y los periodos semestrales I y II de la gestión 2026.

6 Módulos de Dominio y Lógica de Negocio

6.1 Autenticación y Control de Acceso (RBAC)

El flujo se apoya en el DTO de inicio de sesión, el servicio de autenticación y la estrategia Passport JWT. La aplicación cuenta con `JwtGuardia` para validar tokens de sesión y `RolesGuardia` para verificar autorizaciones por rol mediante el decorador `@Roles(...)`. El decorador `@UsuarioActual()` inyecta la entidad autenticada en los controladores.

6.2 Gestión de Usuarios y Becarios

La creación de becarios se desacopla de la de usuarios comunes: el servicio `BecariosServicio` ejecuta de manera atómica el alta de la cuenta y la creación de la ficha académica, verificando que el supervisor asignado exista, se encuentre activo y posea privilegios suficientes. Los supervisores acceden únicamente a la lista de estudiantes bajo su tutela, mientras que el endpoint `/becarios/me` permite al propio becario consultar su información.

6.3 Control de Asistencia y Geolocalización

`AsistenciaServicio` gestiona la presencia física:

- **Ingreso:** Calcula la distancia esférica frente a la sede asignada mediante Haversine. Incorpora tolerancia para el error reportado por el sensor del teléfono (margen ≤ 10 m) y descalifica de inmediato marcas originadas en ubicaciones simuladas (`mock location`).
- **Salida:** Localiza la jornada abierta del día y calcula la fracción de horas trabajadas con redondeo a dos decimales.

6.4 Liquidación de Pasajes: Centavos Enteros y Regla del Día 24

Para erradicar discrepancias contables por redondeo binario, el cálculo opera en centavos de boliviano:

$$\text{tarifa_centavos} = \text{round}(\text{tarifa} \times 100) \quad (4)$$

$$\text{total_centavos} = \sum_{i=1}^n \text{tarifa_centavos}_i \quad (5)$$

$$\text{devolucion_centavos} = \text{round}\left(\frac{\text{total_centavos} \times 80}{100}\right) \quad (6)$$

El envío de la solicitud a revisión está restringido por la **Regla del Día 24**: si la fecha actual es anterior al día 24 del mes (`dia_del_mes < 24`), la API deniega la transacción mediante una excepción `BadRequestException`. Asimismo, ante un eventual rechazo por parte del supervisor, el sistema impone el registro forzoso de observaciones que fundamenten la decisión.

6.5 Evaluación Integral 360° y Formulario F-03

Digitaliza la evaluación semestral estructurada en cinco dimensiones con ponderaciones formalizadas:

- **Liderazgo en la Iglesia (Formulario F-03):** 25 % (Pastor).
- **Autoevaluación Académica:** 20 % (Estudiante Becario).
- **Evaluación del Mentor / Prácticas:** 25 % (Supervisor Diaconía).
- **Evaluación Escuela de Líderes:** 15 % (Facilitador Pastoral).
- **Evaluación Socioeconómica:** 15 % (Trabajador Social).

El método `calcularCalificacionTotal` calcula la nota ponderada sobre 100 puntos y sobre la escala vigesimal (0 a 20 puntos):

$$C_{100} = \sum_{i=1}^5 \frac{P_i \times W_i}{100}, \quad C_{20} = \frac{C_{100} \times 20}{100} \quad (7)$$

Clasificando el rendimiento en cinco categorías: *Sobresaliente* (≥ 90), *Muy Bueno* (≥ 80), *Bueno* (≥ 70), *En Observación* (≥ 60) e *Insuficiente* (< 60). La tabla 1:1 `evaluaciones_pastor` resguarda en formato JSONB las respuestas cualitativas del formulario eclesiástico F-03.

6.6 Prácticas Preprofesionales y Firma Digital

El módulo de prácticas centraliza la acreditación de horas institucionales requeridas por las universidades. El supervisor consolida el total de horas convalidadas y plasma su firma digital manuscrita en formato Base64 (`firma_digital`), garantizando la validez legal del certificado emitido.

6.7 Dashboard Analítico y Notificaciones

- **Dashboard Analítico:** Agrega métricas de horas totales, porcentaje de presupuesto del 80 % consumido frente al límite de Bs 10,000, serie temporal de 12 meses y ranking de puntualidad ponderando marcaciones dentro de horario y geocerca.
- **Notificaciones:** Administra la bandeja de mensajes reactivos y conteo de alertas pendientes, con soporte para integración con Firebase Cloud Messaging (FCM).

6.8 Motor de Emisión de Reportes PDF (`pdf`)

`PdfServicio` ejecuta una instancia *headless* de Puppeteer que compila documentos PDF vectoriales en formato A4 con estricta fidelidad a las planillas impresas de Diaconía FRIF-IFD, integrando membretes, tokens cromáticos, desglose de calificaciones y el recuadro formal de firma digital de conformidad.

7 Matriz de Endpoints REST de la API

8 Verificación de Calidad y Pruebas Automatizadas

El código fuente satisface las pruebas unitarias y de compilación institucional:

- **Compilación (`npm run compilar`):** Empaquetado exitoso mediante Webpack 5 y Nest CLI sin advertencias ni errores sintácticos.
- **Batería de Pruebas Unitarias (`npm test`):** Satisface las 7 suites de pruebas del proyecto (`dashboard`, `evaluaciones`, `pasajes`, `pdf`, `app`) con 16 pruebas automatizadas aprobadas al 100 %.
- **Higiene de Código:** Cero comentarios básicos innecesarios, arquitectura orientada a objetos con repositorios TypeORM y control estricto de tipos de datos.

Método	Ruta de Red	Rol Mínimo	Acción Funcional
POST	/autenticacion/inicio-sesion	Público	Validación y emisión de token JWT
POST	/usuarios	Administrador	Registro de supervisores y administradores
GET	/usuarios	Supervisor	Directorio de usuarios filtrable por rol
PUT	/usuarios/:id/estado	Administrador	Modificación de estado (activo/inactivo)
POST	/becarios	Administrador	Alta atómica de becario y usuario
GET	/becarios	Supervisor	Listado general o por supervisor
GET	/becarios/me	Becario	Perfil institucional del becario
POST	/asistencia/ingreso	Becario	Registro de ingreso con Fórmula Haversine
POST	/asistencia/salida	Becario	Marcación de salida y cómputo de horas
POST	/pasajes	Becario	Creación de solicitud mensual de viáticos
POST	/pasajes/recorridos	Becario	Inclusión de tramos y recálculo del 80 %
POST	/pasajes/:id/enviar	Becario	Envío formal sujeto a la Regla del Día 24
PATCH	/pasajes/:id/aprobar	Supervisor	Aprobación de solicitud de pasajes
PATCH	/pasajes/:id/rechazar	Supervisor	Rechazo con observación obligatoria
GET	/evaluaciones/.../calificacion	Becario	Cálculo ponderado 360° y nota vigesimal
POST	/practicas	Supervisor	Convalidación de horas y firma digital
GET	/dashboard/estadisticas	Supervisor	Consolidado analítico y ranking
GET	/pdf/evaluacion/:id	Becario	Descarga de reporte oficial certificado